



Software Security

Overall Goal

Dynamically loaded (DL) libraries are shared object (.so) files that are loaded at times other than during the startup of a program. In Linux, DL libraries are not actually special from the point-of-view of their format. They are distinguished, as they are not automatically loaded at program start-up; instead, there is an API for opening a library, and looking up for symbols. In practice, this includes the use of `libdl` API in the `<dlfcn.h>` header. In this project, we explore how to re-implement ELF dynamic loading from scratch.

For pedagogical reasons, the project is divided into a number of challenges in order to help you in your progress. You have carte blanche to organize your code in different files/scripts.

High-Level Overview

To load DL library, you first need to map all its `PT_LOAD` segments into the running process memory. Next, you need to locate the exported/imported symbols in order to call them.

As you may recall from our lectures, segments cannot be randomly loaded into the memory. Indeed, their corresponding headers contain hints about their memory address, memory size, file size, memory rights and alignment. Moreover, once loaded, the calling module should be able to find the called functions, but also the loaded module should find its required external dependencies.

You might wonder « Mais que diable allais-je faire dans cette galère ? ». DL libraries are particularly useful for implementing plugins, as the code is loaded only when it is needed. In addition, they can be used for code obfuscation, since common dependencies analyses are no longer possible. They are also useful for implementing a JIT (just-in-time) compiler within an interpreter for efficiency purposes. You may even extend this basic “dynamic loader” to build a basic “ELF interpreter” that executes programs, thereby bypassing any restriction controlling the access of the “exec” system call. In a nutshell, let us explore dynamic loading for fun and profit.

Environment setup

The first step is to create a gitlab project and add your supervisor to it. Then, do the following:

- Create a shared library with two functions that take no parameter and return a string literal (i.e., `const char *`): `foo_exported()`, and `bar_exported()`.
- Create a program (i.e., a translation unit with a “main” function) that directly calls the shared library functions.
- Compile the program, and make sure that everything is working.

Now, repeat the above steps by adding two functions to the shared library with the same signature as the previous ones: `foo_imported()` and `bar_imported()`. Both functions must call an imported function, i.e., defined not in the shared library, but in the calling program.

Challenge 1: Initialize the project

The major task in this project is to develop a tool called **isos_loader** that takes at least 2 arguments:

1. The elf file (shared library) that will be loaded.
2. A list of strings of length at least 1 representing the functions to be called.

Do the following:

- Create the same shared library as in “Environment setup”.
- Create a header file that declares two functions:
 - `my_dlopen()` that loads a DL library named by its string argument, and returns an opaque “handle” (`void *`) for the loaded object.
 - `my_dlsym()` that takes a “handle” of a loaded object along with a string function name, and returns the address where that function is loaded into memory.
 - Both functions should return “NULL” whenever an error is encountered.
- Create a TU that implements `my_dlopen()` and `my_dlsym()`:
 - Just rely on `dlopen()` and `dlsym` from the `<dlfcn.h>` header and `libdl`.
- Create the **isos_loader** program that leverage these functions to load a shared library before calling its functions. Hint: add the `rdynamic` option when you compile the program.
 - Make sure to properly parse the arguments using `argp` instead of `getopt`.
 - Do not forget to create the `--help` option.
 - Run the program to load your shared library, and call its different functions.

Now that you have a functioning project with 3 TUs, let us prepare it for the evaluation part:

- Download the ‘Makefile’ of the Software Security website.
- Complete the Gitlab CI pipeline with the help with your supervisors.

As you might have already understood, the main goal of the subsequent challenges is to re-implement `my_dlopen()` (challenges 2—5) and `my_dlsym()` (challenge 6—7). In order to simplify the task, from now on, compile your shared library in mode standalone (`-nostdlib`) and make sure to hide its dynamic symbols. In addition, remove `rdynamic`, then comment out `foo_imported()` and `bar_imported()`.

Challenge 2: Preliminary checks in the ehdr

When you load a DL library, the first task is to parse its executable header and perform various checks. Within the project, you will implement various helper functions to parse ELF files. Clearly identifying your need from the beginning will definitely help you.

Do as follows:

- Open the shared library in reading mode and parse the executable header into a structure that you create. Do not rely on `libbfd`, nor `libelf`.
- Make sure that the opened binary is of format `elf`, of architecture 64-bit and dynamic.
- Make sure that the size of your `ehdr` structure is equal to the field `ehsize`.
- Note the number of segments (headers). Make sure that it is strictly greater than zero.

Challenge 3: Find the PT_LOAD segments

Now, let us get interested in segments, especially for `my_dlopen()`. After getting the executable header, **isos_loader** loops over all the program headers in the binary to find all the PT_LOAD segments to be loaded in the memory.

To find the PT_LOAD segments, do the following:

- Define a structure to parse a program header.
 - Make sure that the size of your `phdr` is equal to the field `phentsize` in the `ehdr`.
- Loop over all segment headers to get all the PT_LOAD ones. Hint: the `p_type` field.
- Perform four checks:
 - The DL library has at least one load segment.
 - The first load segment spans over all segment headers.
 - The PT_LOAD segments are in ascending order of `p_vaddr`.
 - The PT_LOAD segments do not overlap.
- Compute the total memory size between the first PT_LOAD and the end of the last PT_LOAD.

Challenge 4: The labyrinth of mmap

After locating all PT_LOAD segments, it is time to load them into memory.

- In order to ensure a continuous mapping, it is important to reserve the whole memory space required to contain all the load segments. Hint: `mmap`.
 - Pick 0x00 as the starting address for the mapping.
 - What is the length of the mapping?
 - For memory protection, we apply the design principle “Fail-safe defaults”. Please refer to System Security if you need to refresh your memory. Therefore, what is the desired memory protection for the mapping?
 - Always make your mapping `MAP_PRIVATE`.
 - No particular initialization is needed for the mapping contents.

Without the flag `MAP_FIXED`, `mmap` interprets the starting address as a hint about where to place the mapping. However, `mmap` might pick a new address that may or may not depend on the hint. This mapping address constitutes the load address of the first PT_LOAD segment. Subsequent segments will be loaded relative to this mapping address in order to respect the initial range layout. To this end, all segments must start at an address that has the same constant offset from their `vaddr` value. Thus, this offset, aka base address, is computed as:

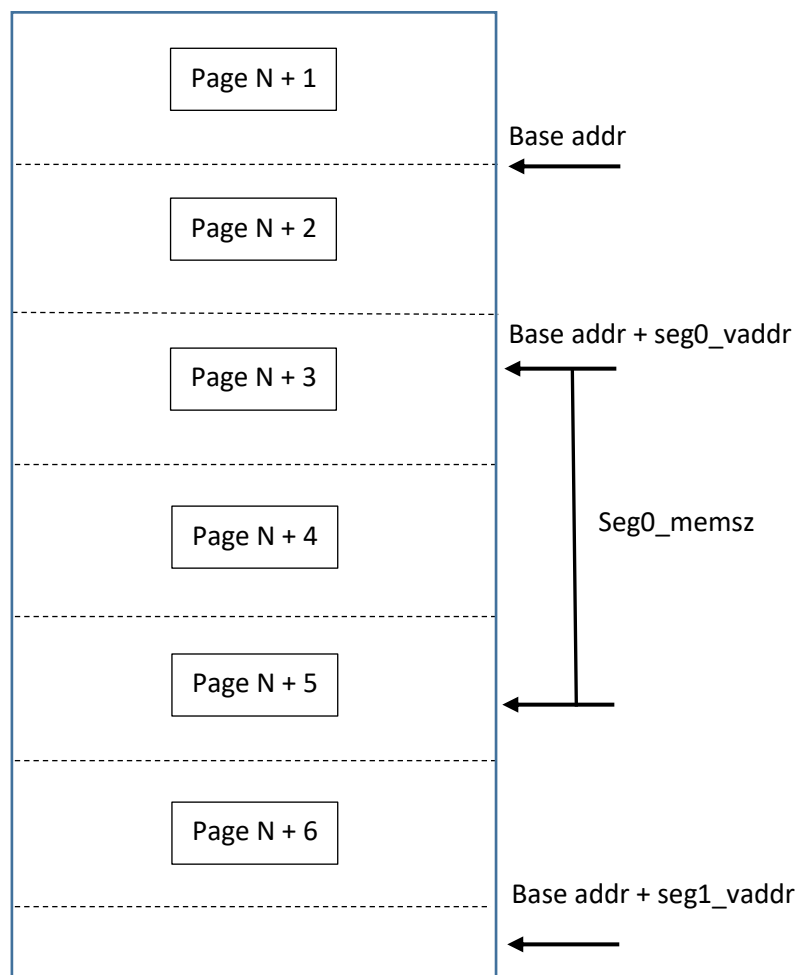
$$\text{base address} = \text{starting address of a segment} - \text{vaddr of the same segment}$$

Unfortunately, in practice, the load address of a segment can be different from its starting address. Indeed, the starting address of a segment does not always begin at page boundaries. Since we can only memory-map at page-aligned addresses, we need to `mmap` some additional bits at the beginning of a loaded segment by rounding down its starting address to the nearest page. Therefore, we have that the segment’s real content starts at: `load address + (vaddr % page size)`. Of course, a rounding down of `p_offset` is also implied to count for the extra mapped bits.

Now you have everything you need to continue.

- Compute the base address of your mapping.
- Loop over the PT_LOAD segments, and load (mmap) each one of them:
 - What is the load address? Hint: think in terms of vaddr and base address.
 - What is the length of the mapping? Hint: remember bss sections.
 - For memory protection, we apply the design principle “Fail-safe defaults”.
 - Do not forget the MAP_FIXED flag. Be careful, the man pages indicate that in this case the load address must be a multiple of the page size.
 - What is the offset of the mapping? Be careful, the man pages also indicate that offset must be a multiple of the page size.
- Loop over the PT_LOAD segments again, and (m)protect each one of them:
 - Use the same load address.
 - What is the length of the mapping? Hint: remember bss sections.
 - What is the desired memory protection for the mapping?

Hint. A picture is worth a thousand hints.



Challenge 5: Dynamic relocations

To complete the process of segments loading, the last step is to relocate the symbols by looking into the `.rela.dyn` section. Do the following:

- Find the address of the `.rela.dyn` section, as well as the number of its entries.
 - Hint: find the segment of type `PT_DYNAMIC`.
- Loop over all entries:
 - Check that its relocation type is `R_X86_64_RELATIVE` (or `R_AARCH64_RELATIVE`).
 - Get the address to be relocated.
- Perform the relocation.
 - Hint: in `*_RELATIVE` entries, the relocation is performed relative to the base address of the loaded segments.

During relocation, you should distinguish two cases depending on whether the address is located in read-only `PT_LOAD` segment. Indeed, relocation requires the `WRITE` permission. Do not forget to reset the access permission after relocation to avoid any security issue.

Challenge 6: Call the exported symbols

Now that segments loading is done, all that remains is to call the DL functions. We will start by calling the self-contained exported symbols, namely `foo_exported()` and `bar_exported()`.

In this project, the design purpose of **isos_loader** is obfuscation: be able to call the hidden API functions (`-fvisibility=hidden`) of some DL library. This would require to craft the DL library in a particular way. Now, proceed as follows:

- In the DL library:
 - Define a structure to be your custom dynamic symbols table. In particular, a symbol table should contain at least the address of the different exported functions.
 - Make this symbol table easy to find for the loader. Hint: what is the only address that you can find in the `ehdr`?
- In the loader:
 - As a final step in `my_dlopen()`, get your dynamic symbol table. Hint: do you need to relocate it?
 - Provide a simple implementation of `my_dlsym()` that returns the address of a function given its name in the dynamic symbol table.
- In your program:
 - Run your program that calls `my_dlopen()` and `my_dlsym()` with different command-line arguments.
 - The moment of truth has come: can you call the functions that you obtain from your dynamic loader? Hint: the answer must be YES.

Challenge 7: Call the imported symbols

To recap, your **isos_loader** can now call functions from a DL library. Nevertheless, the called functions cannot call other imported functions. By imported functions, we mean functions that are defined elsewhere. As you may remember, this requires to resolve these symbols at runtime through the PLT section. In this challenge, we will craft our own. Let's go!

First, uncomment the functions `foo_imported()` and `bar_imported()` in your DL library, and add them to your exported symbol table. Second, define the functions used by them in your program. Try to call any of the `*_imported()` function using Challenge 6. Do you succeed. Hint: the answer must be NO. To make the "yes" win against the "no" (Yes, I quoted both Molière and Raffarin in this project!), we need to manage four components: the imported symbol table in the DL library, the PLT resolving structure in the program, the PLT resolver in the loader, and the custom PLT section in the DL library.

- The imported symbol table in the DL library:
 - Define an integer identifier for each imported symbol.
 - Define an imported symbol table as an array of strings, where the functions names are located in the offset associated to their identifiers.
 - Make this table visible to the loader.
- The PLT resolving structure in the program:
 - Define a structure associating the addresses of the implementation of the imported symbols with their names.
 - Set this structure into the loader handler. Of course, this should be done through a loader API after the call to `my_dlopen()`, since the handler is considered opaque.
 - Define `my_dlset_plt_resolve()` in the loader accordingly.
- The PLT resolver in the loader:
 - Define a function that returns the name of a symbol provided its identifier passed in argument. This function is only concerned about the imported symbol table.
 - Define a resolver that returns the address of a function provided its name passed in argument. Of course, the resolver is only concerned about the symbols inside the PLT resolving structure.
 - Take the `isos-support.h` and `arch` headers from the Software Security website, and add them to your project. Then, copy the code from `isos_trampoline.c` inside your loader.
 - Implement `loader_plt_resolver()` according to your understanding of `isos_trampoline`.
- The custom PLT section in the DL library:
 - Look for the macros `PLT_BEGIN` and `PLT_ENTRY` inside the `isos-support.h`
 - Create a PLT entry for each imported symbol in the DL library.
 - Note that the `PLT_BEGIN` macro uses two undefined symbols: the `loader_handle` as a pointer for an object, and the `isos_trampoline` as a jump address. In order to avoid the famous "segmentation fault", the DL library needs to know the addresses of these symbols that are defined by the loader. Modify both the loader and the DL library accordingly.

Moment of truth #2: make your program successfully run the `*_imported()` functions.

Congratulations! Your journey of dynamic loader ends here. The teaching team hopes that you enjoy all the way. Looking forward to meeting you again in another journey of binary analysis. Until then, keep hacking!

Bonus #1: The GOT Table

Until now, the loader is called each time the DL library calls an imported function. This is not optimal, since the resolved addresses do not change between subsequent calls. Therefore, a better implementation would be to save the resolved address somewhere in the memory after the first call to use it later. This somewhere has a name: the GOT table.

The goal of this bonus challenge is to implement this feature through some custom GOT table. Hint #1. look at the bonus macros in `isos-support.h`. Hint #2: make sure that you modify the `pltgot_entries[id]` in the loader when you are resolving the symbol with "id" as integer identifier.

Bonus #2: Obfuscation

Practitioners tend to forget this fact: the goal of an obfuscation is not to show your biceps by applying the best scrambling techniques, but to hide your game (at least from static analysis). Do as follows:

- Encrypt your DL library with some key.
- Append the encrypted DL library to your program.
- Adapt your program, so that the calls will succeed again.

Elements of evaluation

- You have to do challenges in order: from number 1 to number 7.
- Challenge N gives you (N - 1) points. For instance, challenge 4 gives 3 points. Solving all the challenges gives you 21 points. The first bonus gives 5 points, and the second one gives you 4.
- A video (or written) report should be handed at the end of the project. The report does not give you any positive point. A good report gives you 0 (zero) point, and a very bad report will give you -4 (negative) points. The absence of report will give -5 (negative) points. However, an excellent report gives +1.
- Good code gives you 0 (zero) point, and ill-written code will give you -5 (negative) points. Please consider commenting your code, and organizing your program into small chunks with specific goals. No warning should be issued when you compile with:
 - `clang -Wall -Wextra -Wuninitialized -Wpointer-arith -Wcast-qual -Wcast-align`
 - `gcc -O2 -Warray-bounds -Wsequence-point -Walloc-zero -Wnull-dereference -Wpointer-arith -Wcast-qual -Wcast-align=strict`
 - `gcc -fanalyzer`
 - Run the tool `clang-tidy`.
 - Run your code with clang sanitization options (address, memory, ub), and make sure that your program does not crash.
 - All these tests are also performed through the Gitlab CI during a push.
- Late challenges lose points. You get -1 (negative) point for each week after the deadline.
 - Week 1 starts on March 2nd (One day before Hinamatsuri).
 - You should complete Task N before the beginning of Week N + 2.
 - Deadline to validate the technical challenges is the end of week 8.
- Any plagiarism will be sanctioned by "0" in the whole project:
 - Any shared code will result in "0" for all involved students: provider(s) and copier(s).
 - Plagiarism includes the use of any code-generation tool.